



MIRADOR 360°

**(DAS) Documento Arquitectura de Software
Versión 3.0**

Identificación de Documento

Identificación	Grupo 2
Proyecto	Mirador 360°
Versión	3.0
Documento mantenido por	Grupo 2
Fecha de última revisión	22/05/2026
Fecha de próxima revisión	26/06/2026
Documento aprobado por	
Fecha de última aprobación	

Historia de Revisiones

Fecha	Versión	Descripción	Autor
09/04/2026	1.0	Levantamiento de requerimientos	Simón Molina
22/05/2026	2.0	Creacion de diagramas	Simón Molina
26/06/2026	3.0	Prototipado del sistema	Simón Molina

Tabla de Contenidos

1. INTRODUCCIÓN	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA	7
4.5.1. <i>Propósito</i>	7
4.5.2. <i>Diagrama de despliegue</i>	7
4.5.3. <i>Descripción diagrama de despliegue</i>	7
5. REQUISITOS DE CALIDAD	7

5.1. PROPÓSITO	7
5.3. <i>Reglas y criterios de evaluación de calidad</i>	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. <i>Propósito</i>	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	8
7. PROTOTIPO	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8
9. CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT	8

1. INTRODUCCIÓN

1.1. Contexto del Problema

Actualmente nos centraremos en el edificio “El Mirador” el cual se encuentra en un escenario desfavorable para su administración por la mala gestión de cobros y transparencia de este mismo ámbito. Presenta varios problemas importantes, principalmente todo el papeleo se realiza a mano, sin programas ni documentos de acceso online, lo que produce errores humanos, atraso con las cuotas, lo que repercute en una mala gestión del mantenimiento y transparencia en las cuentas de este edificio.

1.2. Propósito

En este documento se dejará por escrito y de forma detallada cómo va a estar armado e implementado este nuevo sistema. Desde el equipo a cargo de la programación hasta el usuario final, dando a entender el funcionamiento y lo que se hará durante el proyecto.

1.3. Ámbito

Mirador 360 consiste en la creación e implementación de una página web tipo responsive para que cada actor del edificio tenga acceso a ella, otorgando los permisos correspondientes a cada uno según la necesidad del problema. En la página estará disponible toda la información correspondiente y de utilidad para llevar una cuenta clara de los ingresos y egresos de dinero, además de una sección de reclamos y sugerencias/solicitudes con la mayor transparencia posible.

1.4. Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
<i>RESPONSIVE</i>	Diseño de desarrollo web donde el sitio se adapta a cualquier tamaño de pantalla, tablet, computadora, móvil, etc.
<i>MVC</i>	Modelo-Vista-Controlador, tipo de patrón de arquitectura de software que separa una aplicación en 3 partes, mejora la organización y mantenimiento del código
<i>Base de Datos o BD</i>	Es donde se almacena toda la información correspondiente al edificio, nombres, cobros, solicitudes, errores

1.5. Resumen ejecutivo

Básicamente, se busca crear e implementar un sistema automático que reemplace al actual para evitar los problemas que se han generado por el deficiente manejo del sistema de cobro. La idea es que la gente pague a tiempo, las cuentas claras para ambas partes y que la administración no pierda la paciencia y el tiempo cobrando cada que ve un moroso, mejorando así la convivencia del edificio.

1.6. Arquitectura del sistema

Trabajaremos con una arquitectura basada en el patrón MVC (Modelo-Vista-Controlador). El cual nos permite separar en bloques la información que queremos que se muestre a los usuarios finales, para que finalmente en la pantalla de usuario se muestre solo la información relevante para ellos. Además mejora el mantenimiento y corrección de errores del sistema.

2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema será una página web en donde cada usuario podrá ingresar con su cuenta previamente creada, usuario-contraseña. Dentro del sistema los usuarios tendrán acceso a su información personal. Por otro lado, la administración tendrá acceso especial para ver los informes detallados del dinero que entra y sale, y saber de forma inmediata los usuarios morosos o cualquier otra información relevante.

2.2. Objetivos del sistema

- Objetivo general: “Desarrollar una solución tecnológica para el edificio *El mirador* para gestionar las ganancias y gastos de este, asegurando un proceso transparente y mejorando la eficiencia de cobros y reparaciones”
- Objetivos específicos :
 - Registrar la información de los usuarios en una base de datos
 - Realizar cálculos y registros de pagos
 - Almacenar las solicitudes y mantenciones del edificio
 - Generar reportes relacionados a deudas o pagos

2.3. Requerimientos funcionales

1. Gestionar el personal: el sistema debe gestionar de manera eficiente todo usuario relacionado al edificio, incluyendo datos como nombre, contacto y/o número de departamento.
2. Gestionar los pagos: el sistema se encargará de almacenar los registros de los pagos y realizar los cálculos necesarios para realizar todas las operaciones de relevancia.
3. Consulta de deudas: el sistema le dará acceso a los administradores para visualizar a usuarios que presenten deudas pendientes.
4. Registro de solicitudes: el sistema debe permitir ingresar a los usuarios las solicitudes o reclamos en relación al edificio.
5. Consulta de estados de solicitudes: el sistema debe permitir a los usuarios realizar consultas con respecto a las solicitudes ingresadas.
6. Gestión de mantenimiento: el sistema debe llevar un registro sobre las tareas de mantenimiento, marcadas como realizadas o pendientes.
7. Notificaciones a usuarios: el sistema debe notificar de manera

oportuna sobre deudas pendientes o información relevante del edificio.

8. Generación de reportes: el sistema debe generar reportes en base a pagos, deudas, solicitudes y mantenimiento del edificio para el administrador.

2.4. Requerimientos no funcionales

1. El sistema debe registrar de manera segura la información de los usuarios.
2. El sistema debe permitir iniciar sesión a los usuarios.
3. El sistema debe restringir el acceso acorde a los roles del edificio.
4. El sistema debe cerrar sesión de usuarios tras periodo de inactividad.
5. El sistema debe responder a las solicitudes en menos de 2 segundos.
6. El sistema debe soportar a múltiples usuarios simultáneamente.
7. El sistema debe tener una interfaz amigable e intuitiva, fácil de usar.
8. El sistema debe ser de tipo responsive.
9. El sistema debe estar disponible el 99% del tiempo
10. El código del sistema debe permitir modificaciones sin afectar otras funcionalidades.
11. El sistema debe evitar duplicidad de datos y un correcto ingreso de estos.
12. El sistema debe ser compatible con navegadores modernos.
13. El sistema debe ser compatible con diferentes sistemas operativos.
14. El sistema debe enviar notificaciones oportunas.
15. El sistema debe realizar copias de seguridad periódicamente.
16. El sistema debe permitir recuperar datos ante fallos.

2.5. Supuestos y dependencias

- Supuestos: Asumimos que los vecinos tienen acceso a un teléfono o computador con internet para poder meterse al sistema.
- Dependencias: El sistema va a depender de una integración con alguna pasarela de pago externa (como Mercado Pago o Webpay) para poder procesar las transacciones de forma automática y segura.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado

Para este proyecto se utilizará Arquitectura Monolítica en Capas , ejecutado mediante un modelo Cliente-Servidor. Siguiendo los lineamientos de la norma ISO/IEC/IEEE 42010 para la descripción de arquitecturas de software.

3.2. Justificación del estilo según el contexto del sistema

Este estilo organiza el sistema en una jerarquía de capas, donde cada capa tiene una responsabilidad específica y se comunica únicamente con la capa inmediatamente inferior. Además facilita el encapsulamiento de la complejidad del cálculo y procesamiento de los gastos que realiza el software, así también, ocultando la capa por ejemplo de la base de datos en la capa de presentación. Esto ayuda a reducir errores en cadena, permitiendo modificar módulos específicos evitando comprometer la estabilidad del sistema.

Esta arquitectura promueve una alta cohesión, donde cada capa tiene una responsabilidad única y bien definida, y un bajo acoplamiento, minimizando las dependencias directas entre componentes.

Se evaluó la posibilidad de utilizar arquitectura de microservicios por la integración de plataformas externas (para pagos y notificaciones), sin embargo, dada la escala del proyecto, es decir la cantidad de usuario a la que nos enfrentamos, los microservicios agregarían una complejidad operacional e infraestructural innecesaria. La arquitectura en capas centraliza el procesamiento en una única base de datos transaccional

3.3. Patrones de diseño aplicados

Para el caso de Mirador 360 establecimos un patrón de diseño MVC y repositorio, para así poder separar lo que es la interfaz de la lógica, además abstraer la persistencia y centralización del acceso a los datos.

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

El objetivo de este diagrama es dar a conocer gráficamente el comportamiento del sistema, validando su estructura y cómo satisface los requerimientos desde una perspectiva como usuarios finales. Es el que unifica y da sentido a las demás vistas.

4.1.2. Actores

Actores diagrama general caso de uso:

- Administración
- Residente (dueño/arrendatario)
- Personal del edificio
- Sistema de pago
- Sistema de notificaciones
- Comité administrativo

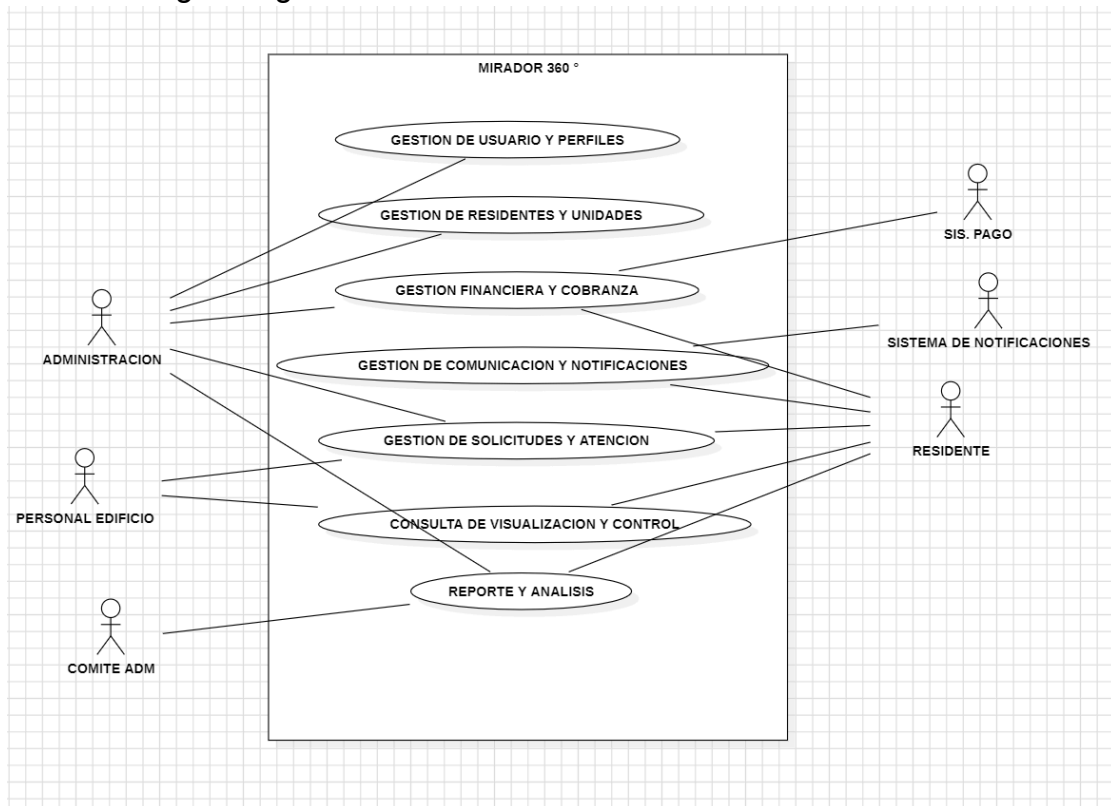
Actores diagrama intermedio caso de uso:

- Administración
- Residente (dueño/arrendatario)
- Sistema de pago

Actores diagrama específico caso de uso:

- Administración
- Residente (dueño/arrendatario)
- Sistema de pago

4.1.3. Diagrama general de casos de uso



4.1.4. Diagrama de casos de uso intermedio y específico

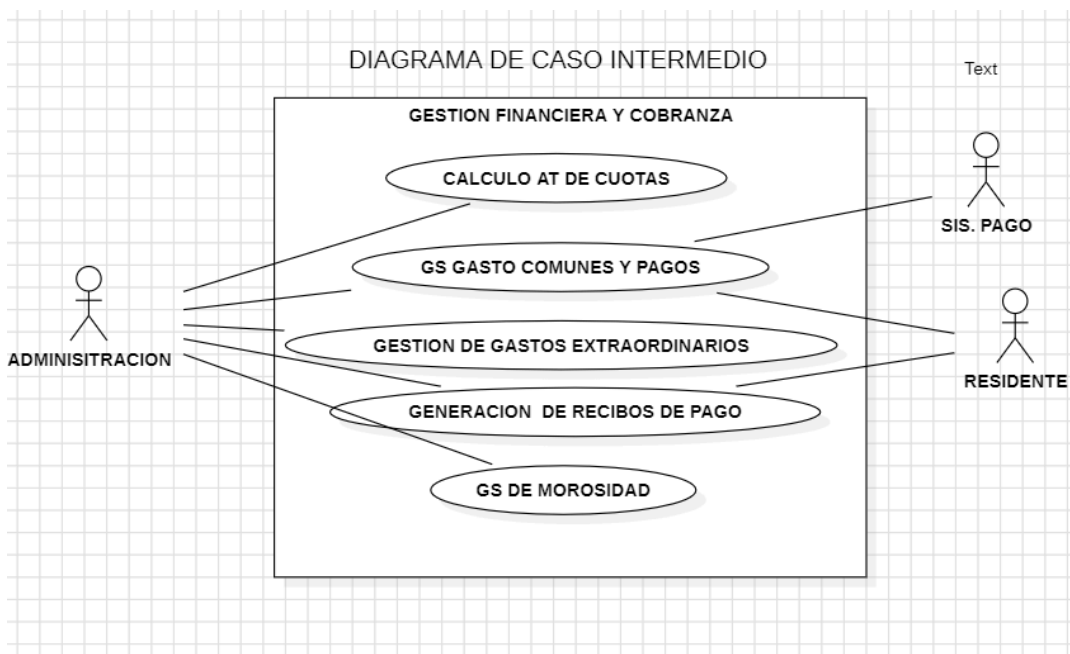
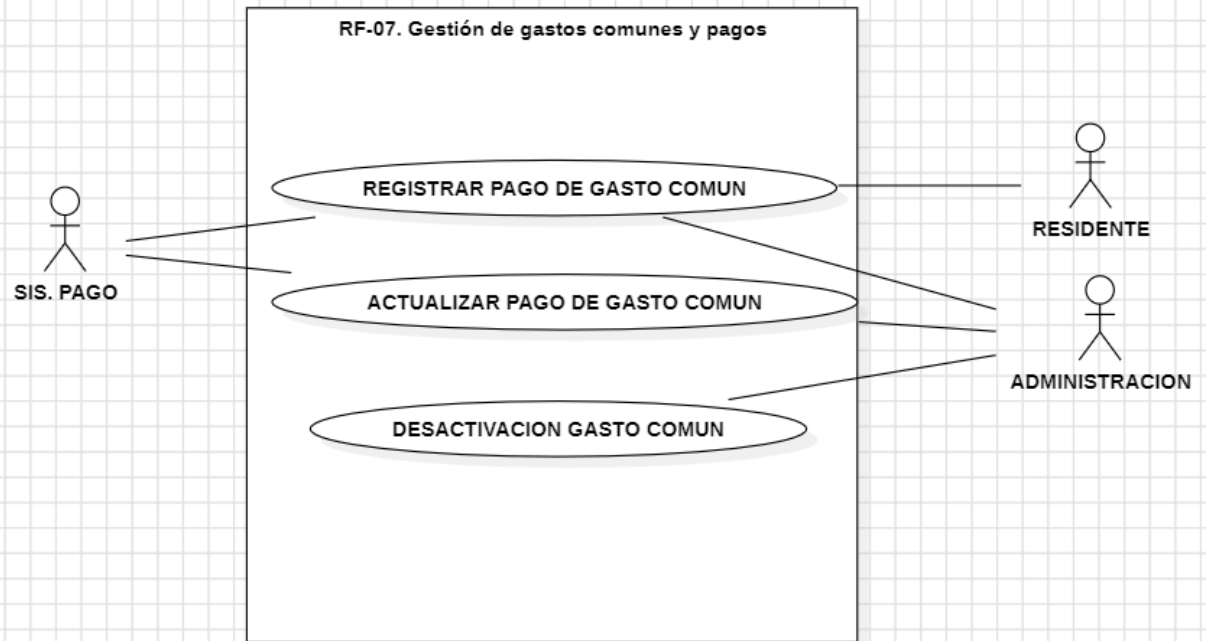


DIAGRAMA CASO ESPECIFICO



4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-001-001	GESTIÓN DE USUARIO Y PERFILES	administración
CU-001-002	GESTIÓN DE RESIDENTES Y UNIDADES	administración
CU-001-003	GESTIÓN FINANCIERA Y COBRANZA	administración, sis. pago, residente
CU-001-004	GESTION DE COMUNICACION Y NOTIFICACIONES	sis. notificaciones, residentes
CU-001-005	GESTIÓN DE SOLICITUDES Y ATENCIÓN	administración, personal edificio, residente
CU-001-006	CONSULTA DE VISUALIZACIÓN Y CONTROL	residente personal edificio
CU-001-007	REPORTE Y ANÁLISIS	administración, comité edificio, residente
CU-002-001	CALCULO AT DE CUOTAS	administración
CU-002-002	GS GASTO COMUNES Y PAGOS	administración, sis. pago, residente
CU-002-003	GESTIÓN DE GASTOS EXTRAORDINARIOS	administración
CU-002-004	GENERACIÓN DE RECIBOS DE PAGO	administración, residentes
CU-002-005	GESTIÓN DE MOROSIDAD	administración
CU-003-001	REGISTRAR PAGO DE GASTO COMÚN	sis. pago, residente, administración
CU-003-002	ACTUALIZAR PAGO DE GASTO COMÚN	sis. pago, administración
CU-003-003	DESACTIVACIÓN GASTO COMÚN	administración

4.1.6. Especificación de casos de uso

Caso de Uso	Gestión de gastos comunes	Identificador:
--------------------	---------------------------	-----------------------

	y pagos	CU-002-002
Actores	administración - sistema de pago - residente	
Tipo	Caso de uso específico	
Referencias	-Gestión de residentes y unidades -Gestión financiera y cobranza	
Precondición	- El Residente debe estar autenticado en la plataforma MIRADOR 360° -El Administrador debe haber ejecutado previamente el proceso de "Emisión de Cobros" para el periodo en curso. -El sistema debe tener conexión activa con la API de la Pasarela de Pagos externa.	
Postcondición	La deuda del residente queda en \$0 para el periodo seleccionado, se actualiza la deuda y existe un registro del pago asociado al departamento.	
Descripción	Describe el flujo integral que permite a un Residente consultar el estado financiero de su departamento	
Resumen	El objetivo fundamental de este módulo es automatizar el ciclo financiero del edificio, garantizando transparencia, integridad de datos y facilidad de uso para los residentes, lo que impacta directamente en la reducción de la tasa de morosidad.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Residente	El residente accede al módulo "Mis pagos"
2	Sistema	El sistema consulta en la BD, calculando el total de deuda incluyendo morosidad si aplica
3	Residente	El residente selecciona las deudas que desee pagar
4	Sistema	El sistema genera un ID de transacción único y redirige al servicio de pago externo
5	Residente	El residente ingresa con credenciales bancarias y confirma el pago
6	Sistema	El servicio de pago retorna el ID exitoso
7	Sistema	El sistema valida el ID y registra los pagos realizados
8	Sistema	El sistema envía mensaje de éxito agregando una opción de descargar comprobante

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
5.1	Pago rechazado por entidad bancaria, el sistema envía al usuario nuevamente a la selección de deudas y solicita que realice el proceso nuevamente.
5.2	Se pierde conexión con el sistema de pago, el sistema envía al usuario nuevamente a la selección de deudas y solicita que realice el proceso nuevamente.

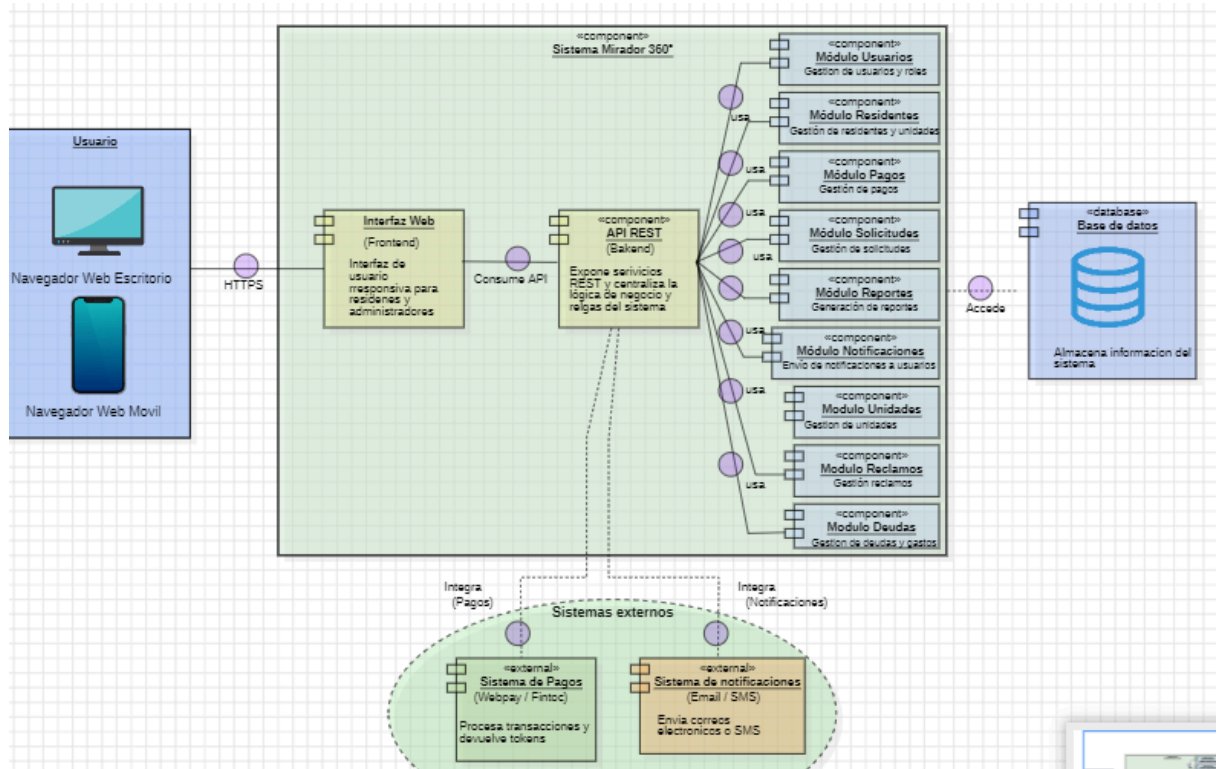
También se incluyen enumeraciones para representar estados y tipos específicos, permitiendo mejorar la organización y mantenibilidad del modelo.

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

4.3.1. Propósito

El diagrama representa la arquitectura modular del sistema y la interacción entre los distintos componentes internos y externos o servicios. Además permite visualizar cómo se organiza funcionalmente la solución y cómo se comunican sus módulos principales.

4.3.2. Diagrama de componente (TODO EL PROY)



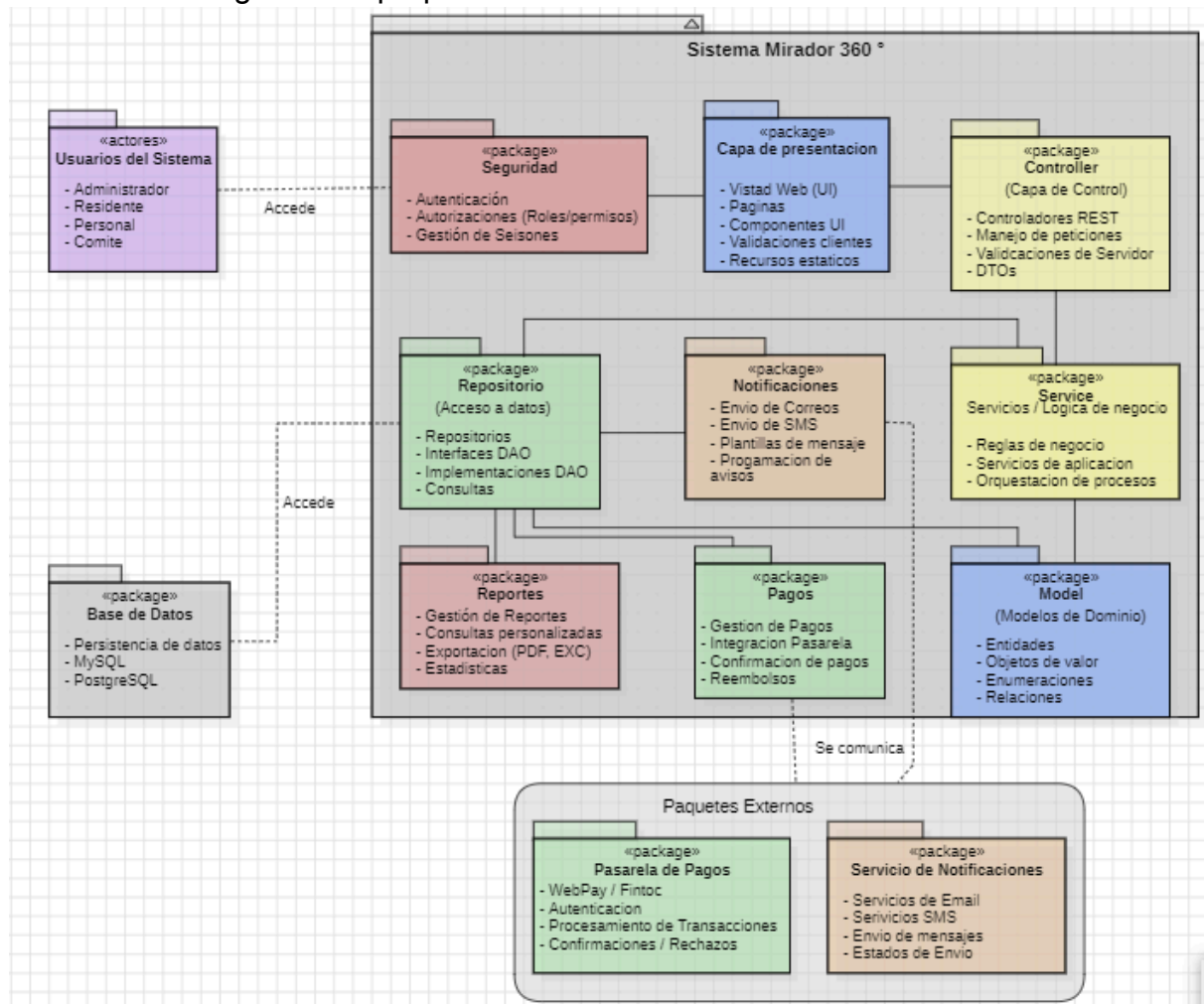
4.3.3. Descripción diagrama de componente

Como hemos comentado anteriormente el sistema Mirador 360° se basa en una estructura de multicapa, como podemos observar está compuesta por:

- Interfaz Web
- API rest
- Módulos internos
- Base de datos
- Sistemas externos, sistema de pago y notificación

Si hablamos del backend, podemos observar que está representado por una API REST la cual comunica cada módulo en donde cada uno desempeña tareas específicas y centraliza la lógica y reglas de negocio.

4.3.4. Diagrama de paquete



4.3.5. Descripción diagrama de paquete

Aquí el diagrama se organiza en paquetes, para poder representar la lógica de negocio y mostrar qué tarea realiza cada uno además de cómo se comunican.

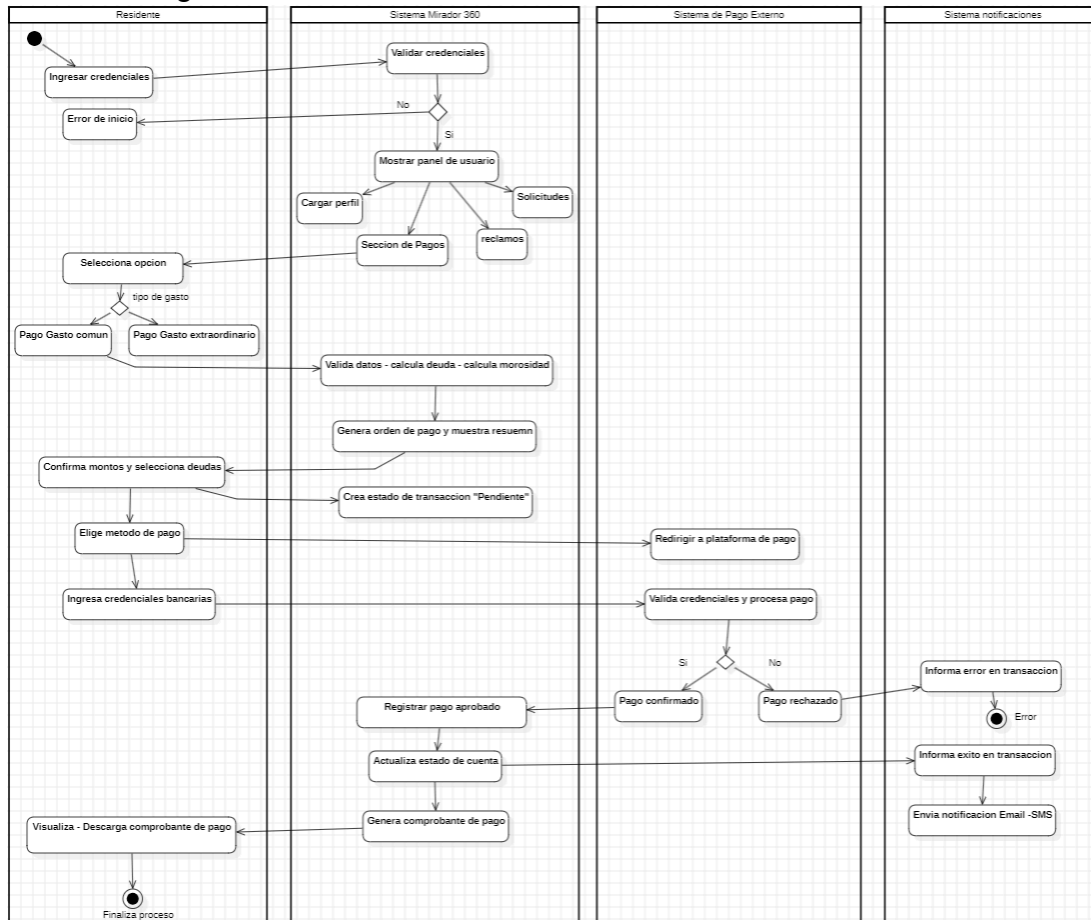
Cada paquete agrupa funcionalidades relacionadas, administrar las solicitudes y respuestas del sistema, representar las entidades del dominio, modelar las integraciones externas y gestionar el acceso a los datos.

4.4. VISTA DE PROCESOS

4.4.1. Propósito

La Vista de Procesos describe el comportamiento dinámico del sistema mediante el diagrama de actividad del caso de uso específico seleccionado, CU-003-02-01, el cual es Registrar Pago de Gasto Común (RF-07). Representa el flujo de trabajo con sus acciones, decisiones y ramificaciones.

4.4.2. Diagrama de actividad



4.4.3. Descripción diagrama de actividad

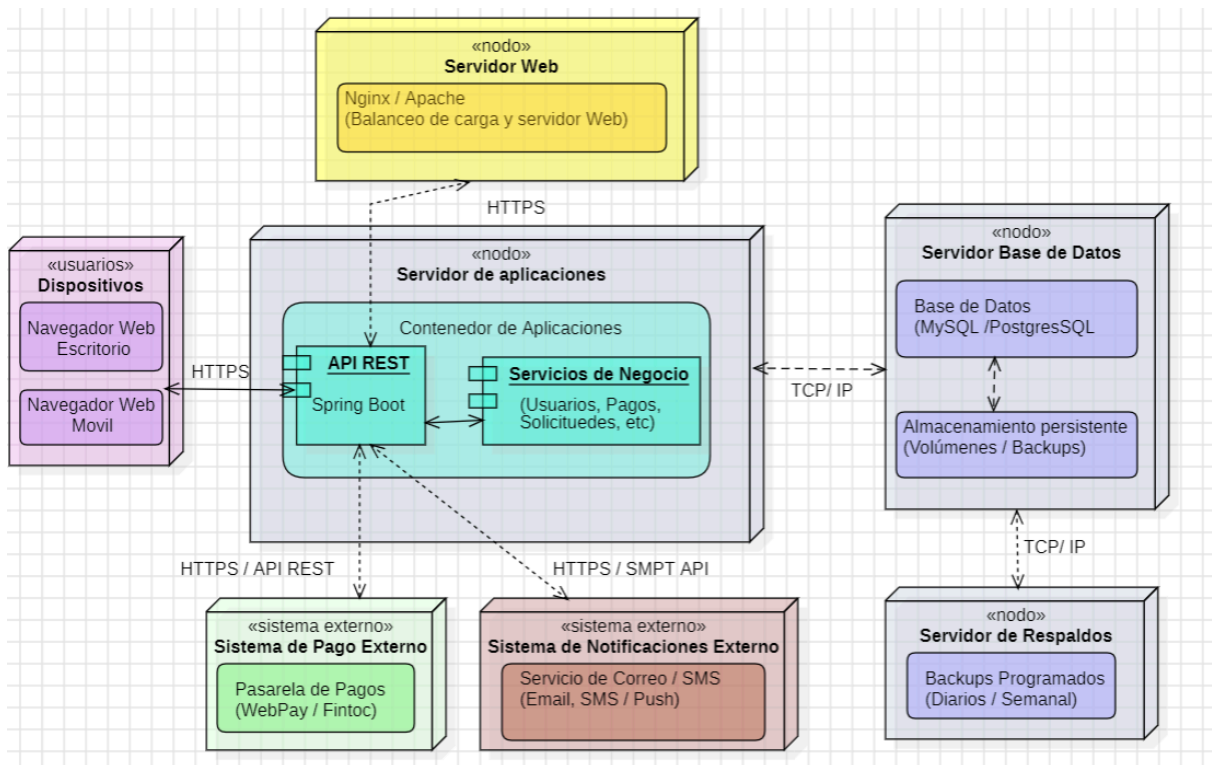
El proceso inicia con el primer paso que es realizado por el residente, donde ingresa sus credenciales y el sistema las debe validar, de ser erróneo entrega un error de inicio. Luego, si es válido, se muestra el panel de usuario en donde se selecciona la sección de pagos y se escoge el tipo de pago a cancelar. Posteriormente el sistema realiza los cálculos pertinentes y el usuario selecciona las deudas a cancelar. Se redirige al servicio externo de pago y este validar sus credenciales, si el pago es exitoso, se actualiza el estado de la cuenta del usuario, en paralelo envía notificación, y entrega la orden de pago para opcionalmente la descarga de comprobante, de ser invalidadas las credenciales, se envía notificación de error.

4.5. VISTA FÍSICA

4.5.1. Propósito

En la vista física nos enfocamos en mostrar la infraestructura y tecnología como tal en donde se ejecutará el sistema. Se pueden visualizar los nodos

4.5.2. Diagrama de despliegue



4.5.3. Descripción diagrama de despliegue

En el despliegue se contemplan los usuarios, los servidores de aplicación, las bases de datos y sistemas externos.

El servidor se conecta con los usuarios mediante HTTPS y así mismo con los sistemas externos, implementando API REST para pagos y SMTP API para las notificaciones. También se muestran los servidores de respaldo para la base de datos, en donde se encuentran backups programados, para respaldar completamente la información valiosa del sistema.

5. REQUISITOS DE CALIDAD

5.1. Propósito

La finalidad de los requisitos es garantizar que el sistema Mirador 360° entregue una solución segura, confiable, mantenible y eficiente para todos los vecinos y actores del edificio. Permitiendo definir los criterios técnicos y funcionales que aseguren la mejor experiencia de usuario posible, adecuándose a las necesidades de cada representante del sistema, además de facilitar la escalabilidad y mantenimiento a futuro de este.

El cumplimiento de los requisitos de calidad permitirá un funcionamiento estable, seguro y organizado, considerando la gestión de usuarios, departamentos, pagos, reclamos, solicitudes y notificaciones.

5.2. Atributos de calidad

ATRIBUTO DE CALIDAD	DESCRIPCIÓN	JUSTIFICACIÓN
Usabilidad	Un sistema intuitivo, fácil de usar y comprender cada sección	Este será utilizado por distintos usuarios, con conocimiento básico de aplicaciones, por lo que es necesario una interfaz que facilite la interacción usuario/sistema
Rendimiento	El sistema debe tener tiempos de respuestas rápidas ante solicitudes de usuarios	Los procesos deben ejecutarse de forma eficiente para garantizar una buena experiencia de usuario para satisfacer sus necesidades
Seguridad	El sistema debe mantener segura la información de usuarios, como información crítica	La plataforma maneja datos sensibles principalmente para los residentes, deudas, transacciones. Se implementará un acceso seguro y validación de datos
Mantenibilidad	El sistema debe facilitar futuras mejoras o modificaciones de todo ámbito	La arquitectura modular y el uso de separación por capas permite modificar funciones o módulos sin afectar todo el sistema
Escalabilidad	El sistema debe permitir añadir nuevas funciones o servicios	La arquitectura modular del sistema permitirá incorporar nuevas funcionalidades y servicios externos sin afectar significativamente los módulos existentes.
Disponibilidad	El sistema debe mantenerse operativo y accesible para usuarios	Los usuarios del sistema requieren acceso a la información constantemente
Portabilidad	El sistema debe poder utilizarse desde distintos dispositivos y navegadores (responsive)	Mirador 360° considera el acceso desde distintas plataformas y dispositivos, por lo cual debe ser adaptativo

5.3. Reglas y criterios de evaluación de calidad

Usabilidad

Reglas y criterios

- El sistema debe presentarse con una interfaz de manera ordenada y fácil de usar.
- Los usuarios deben acceder a las funcionalidades en menos de 3 clics
- El sistema debe ser intuitivo y consistente

Métodos de evaluación

- Pruebas heurísticas de usabilidad
- Validación funcional con usuarios simulados
- Evaluación de navegación y experiencia usuario

Seguridad

Reglas y criterios

- El acceso al sistema requerirá autenticación mediante correo y contraseña
- Cada usuario se le asignará permisos asociados a su rol dentro del sistema
- La comunicación entre cliente y servidor será mediante HTTPS
- Los pagos serán procesados mediante servicios externos seguros
- El almacenamiento de información será encriptado

Métodos de evaluación

- Pruebas de autenticación
- Asignación y validación de controles de acceso por rol
- Revisión de conexiones seguras y acceso a información sensible
- Servicios de pago externos confiables

Rendimiento

Reglas y criterios

- El tiempo de respuesta de funciones principales no debe superar los dos segundos
- La carga del panel principal y consulta de deudas deben ejecutarse de manera eficiente
- La conexión con la pasarela de pago no debe superar tiempo de respuestas

- elevados
- El sistema debe soportar múltiples consultas en simultáneo

Métodos de evaluación

- Pruebas de carga
- Simulación de múltiples consultas
- Medición de tiempos de respuesta
- Validación de procesamiento de pagos

Mantenibilidad

Reglas y criterios

- El sistema debe mantener su arquitectura modular basada en capas
- Cada módulo deberá tener responsabilidades específicas y desacopladas
- La estructura del sistema deberá permitir futuras modificaciones

Métodos de evaluación

- Revisión de diagramas UML
- Inspección de estructura de paquetes y componentes
- Validación de separación entre frontend, backend y servicios
- Evaluación de cohesión y acoplamiento entre módulos

Escalabilidad

Reglas y criterios

- La arquitectura modular del sistema permitirá incorporar nuevas funcionalidades e integraciones externas sin afectar significativamente la estructura existente.
- El sistema deberá facilitar futuras ampliaciones de la plataforma mediante módulos desacoplados.

Métodos de evaluación

- Revisión arquitectónica
- Validación de modularidad
- Simulación de incorporación de nuevos módulos o servicios

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

Todo los software presentan una arquitectura detrás, la cual es parte de la construcción de un sistema, no basta solo con que este funcione, también debe tener características que lo hagan perdurar en el tiempo y facilita la mantención y seguridad de este. Aquí se muestran los distintos principios que componen la arquitectura de un software

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones
Abstracción	Se enfoca solo en los datos relevantes del problema, ignorando los detalles innecesarios del mundo real.	El modelo de datos captura únicamente la información esencial del negocio (como montos y estados), omitiendo detalles irrelevantes de los usuarios.
Bajo Acoplamiento	Los módulos son altamente independientes, evitando que un cambio en uno rompa el funcionamiento de otro.	Los módulos operan con alta independencia, permitiendo integrar o cambiar servicios externos (como la pasarela de pagos) sin alterar el código central.
Modularidad	Divide un sistema complejo en partes más pequeñas, manejables e intercambiables.	La arquitectura está segmentada en bloques independientes (ej. finanzas, login), evitando que el fallo o mantenimiento de uno detenga todo el sistema.
Encapsulamiento	Oculto la información interna de un objeto y permite su modificación solo a través de métodos seguros.	Los datos financieros sensibles están protegidos y su estado solo puede ser modificado a través de procesos transaccionales seguros y validados.

7. PROTOTIPO

7.1. Propósito

El prototipado de un proyecto tiene como principal objetivo dar el primer acercamiento a lo que sería el producto “tangible”, lo que consiste en crear una especie de maqueta con la cual el usuario interactúa, visualiza y valida ideas para asegura que satisfaga las necesidades de este y evitar costosos cambios antes de desarrollar la versión final de este.

7.2. Mockups (imágenes con una breve descripción)



Esta es la pantalla de bienvenida, aquí se encuentra el logotipo del proyecto junto con un mensaje de bienvenida y un slogan. Esta página cuenta con 2 botones, el de comencemos e iniciar sesión, ambos redirigen al inicio de sesión.

 **Mirador 360°**

 Usuario o contraseña incorrectos. Inténtalo de nuevo.

Usuario

usuario

Contraseña [¿Olvidaste tu contraseña?](#)

.....

Iniciar Sesión

[¿Problemas para acceder? Contactar a administración](#)

Esta es la pantalla de inicio de sesión, aquí el usuario debe ingresar con sus credenciales (correo / contraseña) y dar click en “Iniciar Sesión”, existen dos apartados fuera de funcionamiento, la opción para recuperar contraseña y contactar a administración.

 **Mirador 360°**

 **Mi Perfil**

 **Mis Pagos**

 **Solicitudes/Reclamos**

[Cerrar Sesión](#)



Juan Pérez

Depto 402B • Residente

 **Generar Reportes**

Descarga reportes de tu cuenta en formato PDF o Excel.

 **Resumen mens...**

Gastos del periodo...

 **Historial anual**

Comparativa 2025...

 **Estado de cuenta**

Detalle de movimi...

Resumen de cuenta

Total a pagar:

\$150.000



 Gasto Común (Mayo)

\$120.000

 Fondo de Reserva

\$30.000

Ver detalle y pagar →

Posteriormente al iniciar sesión se muestra el perfil de usuario, en donde está su nombre y departamento asociado, junto con su rol. En el menú lateral izquierdo podemos desplazarnos a través de las distintas opciones del sistema, tener en cuenta que “Solicitudes / Reclamos” no está en funcionamiento, a diferencia de “Mis pagos”. Dentro del mismo perfil se puede acceder a los pagos en el botón “Ver detalle y pagar”.

Mi Perfil

Mis Pagos

Solicitudes/Reclamos

Cerrar Sesión

Mis Pagos y Deudas

Revisa tu historial y selecciona las cuotas que deseas pagar.

Filtrar

☒ SELECCIONAR TODO	PERIODO	DESCRIPCIÓN	MONTO	ESTADO	COMPROBANTE
DEUDAS PENDIENTES					
☒	Mayo 2026	Gasto Común	\$120.000	Pendiente	
☒	Mayo 2026	Fondo de Reserva	\$30.000	Pendiente	
HISTORIAL DE PAGOS					
☒	Abril 2026	Gasto Común	\$115.000	Pagado	Descargar

Total seleccionado: \$150.000

Ir a Pagar

Este es el menu de pagos, aqui se visualizan cada una de los pagos y deudas del residente, indica la fecha, una descripcion, el monto, estado de la cuota y en el caso de estar pagada un boton para descargar comprobante de pago. Dentro de esta misma ventana se pueden seleccionar los pagos los cuales quieres cancelar y luego dirigirte al boton de “Ir a pagar”. Ademas cuenta con una opcion para filtrar las cuotas por fecha y estado.

Solicitudes/Reclamos

Mis Pagos y Deudas

Revisa tu historial y selecciona las cuotas que deseas pagar.

Filtrar

☒ SELECCIONAR TODO	PERIODO	DESCRIPCIÓN	MONTO	ESTADO	COMPROBANTE
DEUDAS PENDIENTES					
☒	Mayo 2026	Gasto Común	\$120.000	Pendiente	
☒	Mayo 2026	Fondo de Reserva	\$30.000	Pendiente	
HISTORIAL DE PAGOS					
☒	Abril 2026	Gasto Común	\$115.000	Pagado	Descargar

Total seleccionado: \$150.000

Ir a Pagar

Filtros

ESTADO

Todos Pendiente Pagado

MES

Ene Feb Mar

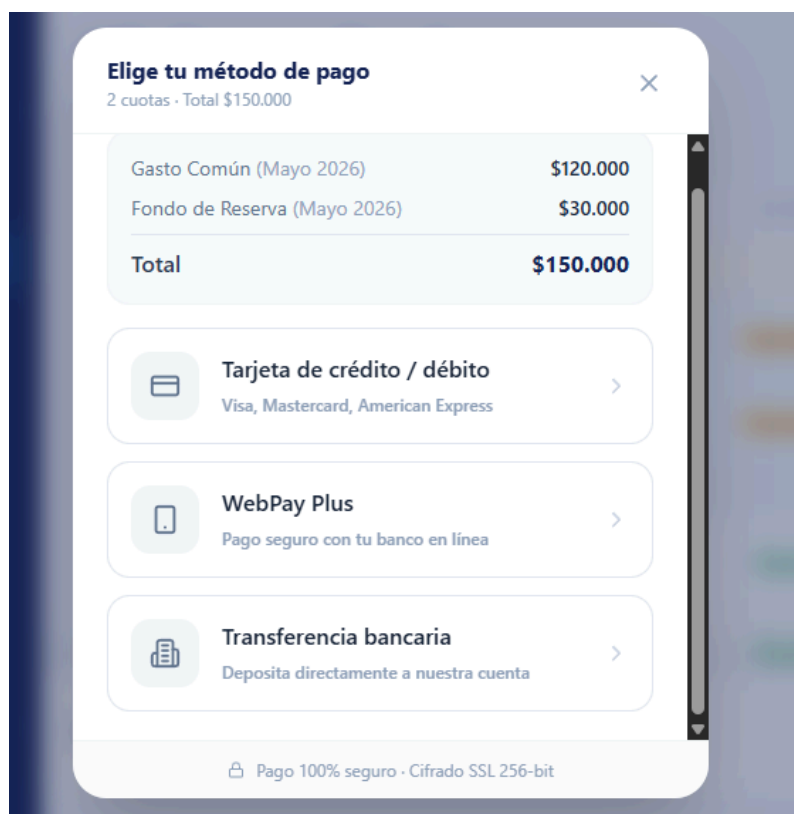
Abr May Jun

Jul Ago Sep

Oct Nov Dic

AÑO

2026



Al momento de pagar una o mas cuotas seleccionadas se abre este menu, en donde el residente elige el método de pago mas conveniente para el, las opciones disponibles son, tarjeta de credito / debito, WebPay Plus o transferencia bancaria.

← Tarjeta de crédito / débito

Gasto Común (Mayo 2026) \$120.000
Fondo de Reserva (Mayo 2026) \$30.000
Total \$150.000

NÚMERO DE TARJETA
0000 0000 0000 0000

NOMBRE DEL TITULAR
Como aparece en la tarjeta

VENCIMIENTO CVV
MM/AA ...

Pago 100% seguro · Cifrado SSL 256-bit

← WebPay Plus

WebPay Plus — Transbank

Serás redirigido al portal seguro de Transbank para completar tu pago con la clave de tu banco.

Transacción cifrada y certificada por Transbank. Tus datos bancarios nunca son compartidos con nosotros.

Total a cobrar: **\$150.000**

Continuar con WebPay >

Pago 100% seguro · Cifrado SSL 256-bit

← Transferencia bancaria

Cuenta corriente N°
12345678-9

RUT titular
76.543.210-K

Nombre
Administración Mirador 360°

Monto exacto
\$150.000

Correo (comprobante)
pagos@mirador360.cl

Copiar todos los datos Copiar todo

Envía el comprobante a pagos@mirador360.cl con tu nombre y número de departamento. La acreditación queda...

Pago 100% seguro · Cifrado SSL 256-bit

En cada una de las opciones se muestran los datos o pasos a seguir para realizar el pago correspondiente.

7.3. Justificar herramientas de prototipado

Para prototipar el proyecto se optó por utilizar una herramienta de trabajo comúnmente utilizada en el mercado, **Figma**, la cual es de acceso gratuito en su primera versión, cuenta con el apoyo de IA, es intuitiva y permite realizar cambios instantáneos sin la necesidad de operaciones de edición compleja, además no es necesario tener conocimientos amplios de HTML (lenguaje ocupado en estructura de páginas web) para trabajar eficientemente con esta herramienta. Entonces, gracias a Figma podemos trabajar abaratando costos y de manera óptima.

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

Para evaluar la calidad de este proyecto nos enfocamos en utilizar dos métodos, los casos de prueba y la evaluación heurística de Nielsen. Esto para asegurar que el sistema no solo “funcione correctamente” sino evaluar que cumpla con ser intuitivo, eficiente y seguro de operar para el usuario final. Mediante los casos de prueba se verifica que el sistema cumpla con los requisitos de negocio y seguir los flujos correctos de manera segura y esto lo complementamos con la verificación basada en los 10 principios de usabilidad de Jakob Nielsen, el cual tiene como objetivo inspeccionar la interfaz desde la perspectiva del residente-

8.2. Lista de verificación

H1. Visibilidad del estado del sistema: No cumple. Los gastos se muestran como un monto total sin ningún tipo de detalle o desglose. Esto hace que sea difícil para los residentes entender bien qué se les está cobrando.

H2. Relación entre el sistema y el mundo real: No cumple. La plataforma solo está en español y no tiene un botón para cambiar de idioma. Esto deja afuera a los residentes extranjeros que solo hablan inglés y les impide usar el sistema para pagar.

H3. Control y libertad del usuario: No cumple. El perfil del usuario está solo para lectura; es decir, no da la opción de actualizar datos básicos de contacto como el correo o el teléfono en caso de que cambien.

H4. Consistencia y estándares: No cumple. Hay una redundancia en la pantalla de inicio: los botones "Comencemos" e "Iniciar Sesión" llevan exactamente al mismo lugar, lo que puede confundir un poco al usuario sobre qué opción elegir.

H5. Prevención de errores: Cumple. En la revisión no se encontraron fallos graves de este tipo que corten o arruinen el proceso principal de pago del residente.

H6. Reconocimiento antes que recuerdo: Cumple. El diseño no obliga al usuario a tener que memorizar información al pasar de una pantalla a otra, por lo que navegar por la plataforma resulta bastante cómodo.

H7. Flexibilidad y eficiencia de uso: No cumple. Falta la típica opción de "Recordar usuario". Al no tenerla, la persona tiene que escribir sus credenciales completas cada vez que entra, lo que hace el proceso más tedioso para quienes usan el sistema seguido.

H8. Diseño estético y minimalista: No cumple. Se usó un tono de texto gris sobre un fondo blanco que tiene muy poco contraste, lo que hace que leer la información sea cansador para la vista.

H9. Ayudar a los usuarios ante errores: Cumple. No se detectaron problemas graves con los mensajes de error ni situaciones en las que el usuario se quede "atrapado" sin poder volver atrás por su cuenta.

H10. Ayuda y documentación: Cumple. El sistema es lo bastante directo e intuitivo para su objetivo principal (pagar los gastos comunes) como para no necesitar manuales o tutoriales extensos en esta etapa.

8.3. Análisis y métricas de resultados

Para evaluar las métricas se procesaron los datos obtenidos en el Registro de problemas de usabilidad, agrupando los fallos según su nivel de severidad y la heurística afectada.

Métricas de severidad

- Problemas Catastróficos (Severidad 4): 0 hallazgos.
- Problemas Mayores (Severidad 3): 2 hallazgos.
- Problemas Menores (Severidad 2): 3 hallazgos.
- Problemas Cosméticos (Severidad 1): 1 hallazgo.
- Total de hallazgos detectados: 6 problemas de usabilidad.

De las 10 heurísticas evaluadas, el prototipo cumple con 4 principios fundamentales (H5, H6, H9 y H10) y presenta requerimientos de mejora en las 6 áreas restantes (H1, H2, H3, H4, H7 y H8). Estos resultados proporcionan una base clara para aplicar correcciones de diseño en la siguiente etapa de desarrollo.

9. CONTROL DE VERSIONES

9.1. Propósito

La finalidad de versionar un proyecto es poder llevar un historial y trazabilidad sobre los cambios que se realizaron, quien los hizo y en qué momento, además permite tener un punto de retorno en caso de cometer algún error durante una modificación y así poder revertirlo.

Además en el trabajo colaborativo es muy útil, para saber quien ha modificado cada cosa o entregar lo solicitado dejando un registro.

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial))

Para el control de versiones en este proyecto utilizamos dos tipos, uno para la documentación y otro para el prototipo de la interfaz.

Cuando hablamos de documentación nos referimos a los diferentes documentos entregados durante el proyecto, en el cual utilizamos una estrategia de versionamiento semántico trabajando con herramientas de control de versiones como GitHub, en donde alojamos los entregables de este proyecto de manera ordenada. Para el caso de la estrategia de versionamiento del prototipo de la interfaz la herramienta Figma ya nos da por determinado un versionamiento secuencial, en donde se inicia con una versión inicial 1 y continúa sucesivamente hasta llegar a la versión final.

9.3. Justificar herramientas de versionamiento

Para el caso de GitHub es una de las mejores opciones para el trabajo ordenado y colaborativo, ya que permite acceder fácilmente a cualquier proyecto con el cual se quiera trabajar, permite que los colaboradores accedan a estos archivos en cualquier momento y lugar de manera instantánea. Además nos permite un trabajo ordenado, manteniendo un seguimiento sobre cada archivo y sus modificaciones correspondientes realizadas por cada colaborador.

Y para Figma, como se mencionó anteriormente es una de las mejores herramientas disponibles para prototipar un proyecto de esta magnitud, la cual cuenta con su propio control de versiones de tipo secuencial.

7. CONCLUSIONES

A modo de cierre de este proyecto, se evidenció cada una de las etapas necesarias para desarrollar un sistema de software, recorriendo los distintos

ciclos de vida del desarrollo. En una primera instancia se identificó el problema principal que se buscaba resolver, junto con los requerimientos del modelo de negocio, para posteriormente seleccionar la metodología de trabajo más adecuada.

Luego se desarrollaron las distintas vistas arquitectónicas, las cuales permitieron representar el funcionamiento del sistema, los actores involucrados en cada caso de uso y los componentes necesarios para garantizar su correcto funcionamiento. Todo este proceso se documentó mediante diversos diagramas que presentan la información relevante para comprender la arquitectura y el diseño del proyecto.

Además se concluyó que un software de calidad requiere cumplir con requisitos no funcionales que aseguren aspectos como la seguridad, la estabilidad, la mantenibilidad y la usabilidad. Estos atributos permiten entregar una solución confiable, organizada y alineada con las expectativas tanto de los usuarios finales como del cliente.

En la última etapa se construyó el primer prototipo usable para mostrar en un inicio como debería funcionar el sistema, se utilizó una herramienta de prototipado en donde se le realizaron diferentes pruebas de ejecución y se le aplicó la evaluación heurística de Nielsen

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt